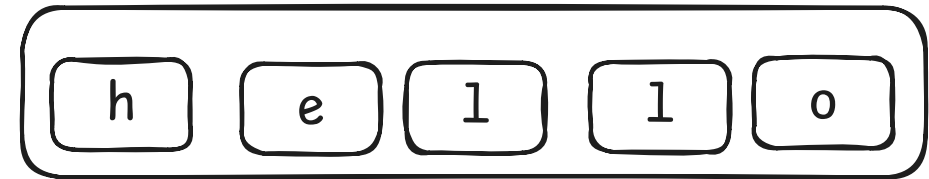


Lecture03 - Strings and Maps

Strings are bytes sequence

```
s := "hello"
```

```
len(s) // 5
```



string

byte

rune

```
s := "Привет"
```

```
len(s) // 12
```

byte — alias —> uint8

A byte 0-255

'A' -> 65

'B' -> 66

'C' ->

```
var b byte = 65
```

```
fmt.Println(b) // 65
```

```
fmt.Printf("%c\n", b) // 'A'
```

```
s := "hello"
```

```
fmt.Println(s[0]) // 104
```



s[index] returns byte

```
s := "Go"
```

```
s[1] = 'H' // error
```

```
s := "Go"  
b := []byte(s)
```

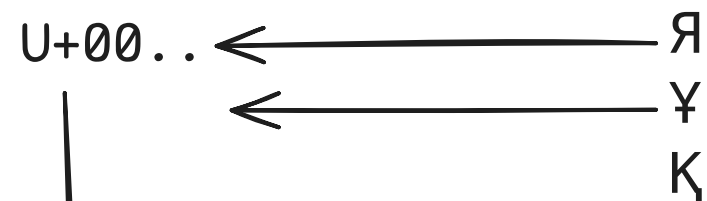
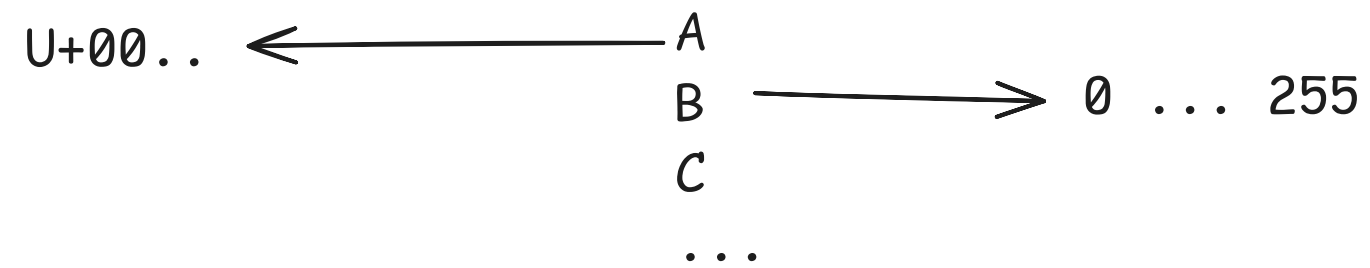
```
fmt.Println(b) // [ASCII(G), ASCII(o)]
```

```
type _strings struct {  
    d *[]byte  
    len int  
}
```



```
s := []byte("Go")  
s[1] = 'H'
```

```
fmt.Println(string(s)) // "GH"
```



represented in Go

rune

byte — alias → uint8

rune → int32

" " -> string

' ' -> rune literal

```
ch := 'Я'
```

```
fmt.Println(ch) // 1... : unicode code point
```

```
fmt.Printf("%c", ch) // Я : character
```

```
fmt.Printf("%U", ch) // U+... : unicode
```

UTF-8

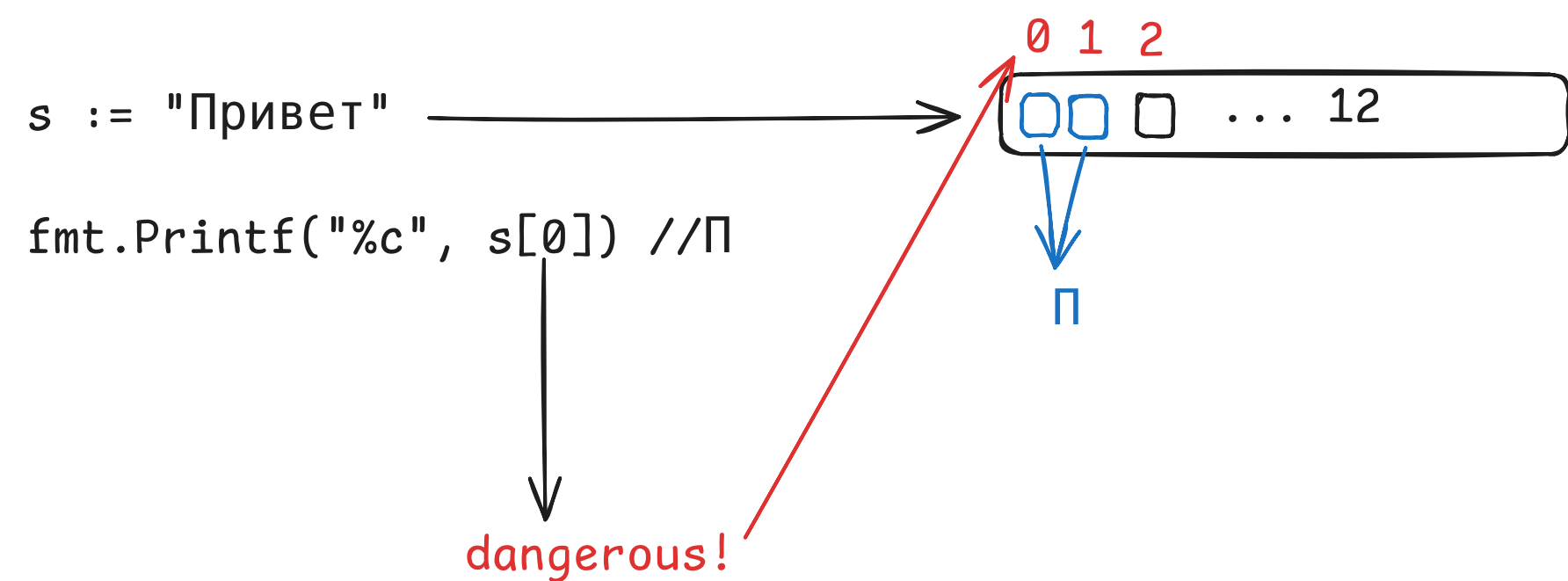
Unicode coding points — store → memory

Unicode coding point -> utf-8 encoding -> some bytes

ASCII : 1 bytes

Cyrillic: 2 bytes

Asian alphabet: 3 bytes



```
s := "Привет"
```

```
len(s) // 12
```

```
utf8.RuneCountInString(s) // number of runes, 6
```

```
for i := 0; i < len(s); i++ {  
    ... // iterate through bytes  
}
```

```
for i, r := range s {  
    // iterate through runes  
}
```

MAP

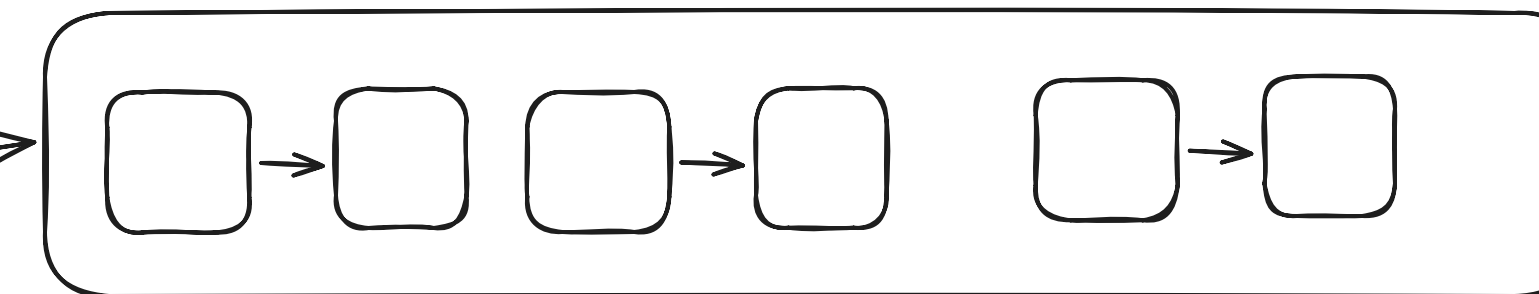
```
names := []string{"Alikhan", "James", "Alexander"}
```

```
grades := []int{0, 100, 50}
```

```
i  
names{}
```

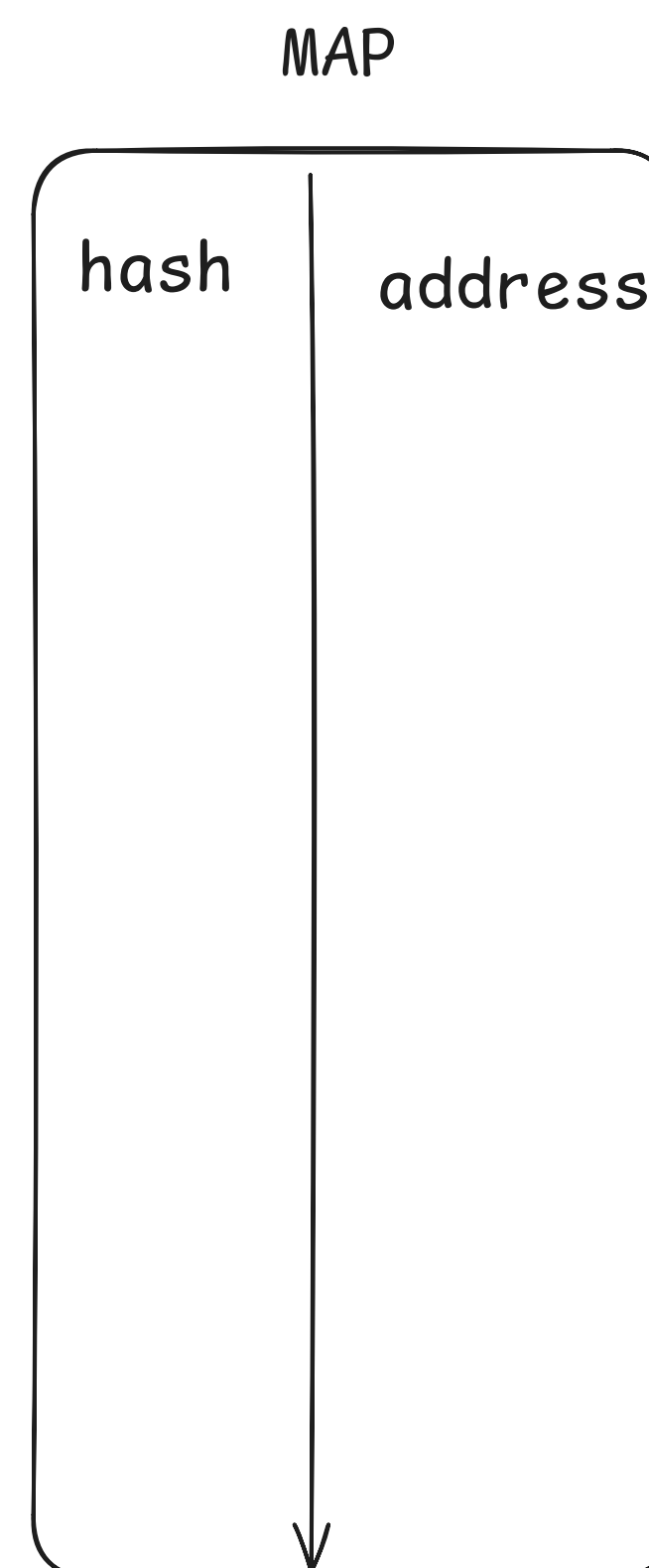
```
j  
grades{}
```

```
grades := map[string]int{
    "Alikhan": 0,
    "James": 100,
    "Alexander": 50,
}
```



```
key --- hash function --> hash,
```

```
grade["Alikhan"] = 100
```



SWISS TABLES

